No Spoiler Please! A TinyBERT Model to Detect Spoiler Reviews on IMDb

Marco Colangelo[1]

[1]Department of Computer Science, University of Milan, Milan, Italy
Corresponding author: marco.colangelo@studenti.unimi.it

Abstract

This report contains the explanation of the work done for the final project of the Natural Language Processing course for the 2025/26 academic year. The project focused on the application of knowledge distillation, a technique used to compress large language models into smaller and more efficient versions. This is achived by training a smaller model (the student) to replicate the behavior of a larger model (the teacher). In particular, this project had the aim to implement a spoiler detector for IMDb's movie reviews: a BERT-base model has been used as the teacher model, while a TinyBERT_General_4L_312D has been used as the student model. The experiments shows that WAITING FOR RESULTS

1 Introduction

Very often, before even seeing a film or a TV series, we decide to read some reviews because we're curious to know the opinions of those who have already seen it. In such a situation, it's not uncommon to encounter reviews that, whether intended to spoil something or not, include spoilers (e.g., "The movie is great, but character X dies/does this or that"). Clearly, we want to avoid this: luckily, platforms like IMDb warn us in advance about such reviews.

IMDb is by far the most popular global source for movies and TV shows, and it also includes user-submitted reviews with spoiler warnings for each one. The problem is that users mark reviews as spoilers themselves, but they can lie and insert spoilers on a no spoiler marked review. Other users can report a review as spoiler, and the IMDb moderation team will label it as such, but of course it's not an immediate action. What if there were a browser extension (running locally on users' PCs) that retrieves the text of a review and tells us "Spoiler alert!" before we can even begin reading it? With the recent evolution of LLMs, this task can be accomplished by fine tuning a sufficiently large pre-trained model, such as BERT.

Language model pre-training, such as BERT, has significantly improved the perfomances of many NLP tasks. However, pre-trained language models are usually computationally expensive, so it is difficult to efficiently execute them on resource-restricted devices (such as the user's PC). A novel Transformer distillation method, specially designed for knowledge distillation of the Transformer-based models, allows to effectively transfer to a smaller student model called TinyBERT the plenty of knowledge from a large pre-trained BERT model. [4]

1.1 Related Work

So far, three main studies have defined the paradigm of model compression via knowledge transfer.

- The first, by Hinton et al. [2], introduced the seminal concept of Knowledge Distillation (KD), focusing on transferring the dark knowledge from a large teacher to a compact student. Their approach minimizes the distance between the output probability distributions of the two networks, softened by a temperature T.

- The second study, by Jiao et al. [4], proposes a variant named TinyBERT trained into two distinct phases: General Knowledge Distillation first and Task-Specific Distillation second. While Hinton's method operates only on the final output layer, Jiao et al. argue that for complex language models, this is insufficient. Their method forces the student to mimic not only the teacher's prediction but also its intermediate representations.
- The third, by Ji & Zhu [3], attempts to provide a mathematical justification for why knowledge distillation works, demonstrating that the use of hard labels (ground truth) are necessary during distillation to counteract the so-called "imperfect teacher". This way, the student can understand how much he can trust the teacher's predictions.

The proposed TinyBERT model diversifies by using only task-specific distillation and introducing a loss factor on hard labels for prediction layer distillation.

2 Research question and methodology

Spoiler detection with LLMs is not a trivial task due to the subtle, context-dependent nature of what constitutes a spoiler, as well as the length of the reviews which can exceeding BERT's 512-token limit.

Research question: *To what extent can a highly compressed language model (TinyBERT) maintain not only the predictive performance of a larger teacher model (BERT-base) but also its internal reasoning and contextualization capabilities in a highly ambiguous task like spoiler detection?*

2.1 Proposed approach

The proposed approach is based on the following pipeline.

- **Data Preprocessing:** a truncation strategy was applied that considers only the first 128 and last 382 tokens of a review's text. This is because spoilers are often found at the end of a review, but the initial part is important because it introduces context. [6]
- **Teacher Fine Tuning:** layer-wise learning rate decay was used to avoid catastrophic forgetting. Also, weighted cross-entropy loss was used due to the dataset's imbalance.
- **Task-Specific Distillation:** this phase is divided into two subphases. In the first, intermediate-layer distillation is performed, while in the second, prediction-layer distillation is performed.
- **Evaluation and computational saving metrics:** some standard plots such as bar charts of evaluation metrics and confusion matrix have been constructed, as well as other plots to evaluate the computational savings of the student model compared to the teacher model.
- **Embeddings and Attention Matrices exploration:** in order to see if the student's internal representations actually aligned with those of the teacher, techniques such as PCA on the embeddings of both models and plots of the attention matrices using heatmaps were used.

3 Experimental Results

3.1 Data and Preprocessing

The dataset used was the IMDB Spoiler Dataset [5], available on Kaggle. This dataset (in JSON format) contains over 500,000 IMDb movie reviews, each with a boolean label that marks it as a spoiler or not. I choose to extract a random sample of 300,000 total reviews, then divided it into training (80%), validation and test sets (10% each).

To overcome the 512 maximum token limit with BERT, a solution based on the head+tail of the review text was used [6]: the text is first tokenized using the BERT vocabulary, also adding the special tokens [CLS] at the start and [SEP] at the end. If the resulting token list is longer than 512, only the first 128 tokens after [CLS] and the last 382 before [SEP] are retained.

Each row, saved in a .parquet file for efficiency, contains *input_ids*, *attention_mask* and *token_type_ids*. If the review exceeds the limit, head+tail truncation is applied to build a 512-token sequence; otherwise the sequence keeps its natural length and is padded dynamically at batch time.

3.2 Teacher and Student Models Architectures

The choosen teacher model was BERT-base-uncased [1], while the choosen student model was TinyBERT_General_4L_312D [4]: this because this is exactly the TinyBERT model from Jiao et al with general knowledge istillation already performed (and therefore not necessary for my work) and, more importantly, it allows to implement the same techniques they use to handle the mismatch in the number of layers and dimensions of hidden states and embeddings [4]. Table 1 shows the differences in the number of parameters between the two models.

Table 1: Parameters of the teacher and student models.

Model	Layers	Attention heads	Embeddings	Hidden states	Total params
BERT-base-uncased	12	12	768	768	110M
TinyBERT-general	4	12	312	312	14.5M

Both the fine tuning of the BERT teacher model and the task-specific distillation of the student were performed on a desktop PC equipped with an RTX 3070 (8GB of VRAM) and 32GB of RAM.

3.3 BERT Fine Tuning for Spoiler Detection

Since the dataset contains an imbalance in labels (73% no spoiler and 26% spoiler), a weighted cross entropy loss (WCE) was used during BERT fine tuning, giving more importance to the spoiler class. The weight has been calculated on the train set as the ratio between the number of negative samples and the number of positive samples.

The hyperparameters used for Teacher fine tuning are the following:

- Epochs: 3
- Batch size: 24 (physical dimension = 8 and gradient accumulation = 3)
- Warmup ratio: 0.03
- Optimizer: AdamW with weight decay of 0.01 and a custom learning rate implemented as Layer-wise Learning Rate Decay (base LR = $2e-5$, decay factor = 0.95) to give a lower LR to the lower layers of the network. This, for BERT fine tuning, can overcome the catastrophic forgetting problem. [6]
- Metric for best model: PR-AUC on validation set. This choice is justified by the imbalance of the dataset and the chosen task.

Figure 1 shows the metrics obtained with the fine tuned teacher compared to the teacher without fine tuning.

3.3.1 Results of BERT Fine Tuning

3.4 Task-Specific Distillation

The task-specific distillation phase is in turn divided into two distinct sub-phases.

3.4.1 Phase 1: Intermediate Layers Distillation

During this subphase, embedding, hidden-state and attention distillation are performed, while the classification head is frozen on both models. This because attention weights learned by BERT

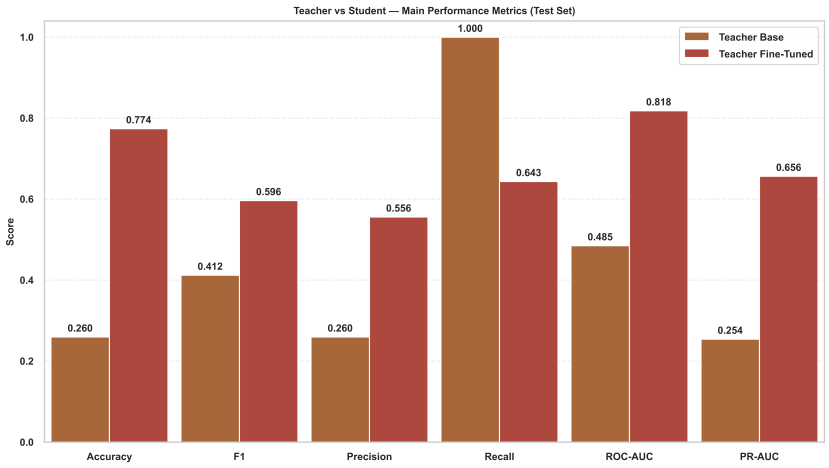

Figure 1: Inserisci qui la didascalia della tua figura.

can capture rich linguistic knowledge and is essential for natural language understanding, so it should be transferred to the student.

More formally, given an input x, the loss for this phase is calculated as

$$L_{TD1}(x) = \lambda_0 L_{embd}(x) + \sum_{m=1}^{M} \lambda_m (L_{hidn}^{(m)}(x) + L_{attn}^{(m)}(x)) \tag{1}$$

where

- $L_{attn} = \frac{1}{h}\sum_{i=1}^{h} MSE(A_i^S, A_i^T)$ is the attention-based distillation. h is the number of attention heads, $A_i \in \mathbb{R}^{l\times l}$ is the unnormalized attention matrix of the i-th head of teacher or student and l is the input text length (512 in this case).
- $L_{hidn} = MSE(H^S W_h, H^T)$ is the hidden states-based distillation. $H^S \in \mathbb{R}^{l\times d'}$ and $H^T \in \mathbb{R}^{l\times d}$ refer to the hidden states of the student (with size $d = 312$) and teacher (with size $d = 768$) respectively, while $W_h \in \mathbb{R}^{312\times 768}$ is a learnable linear transformation of the hidden states of student into the same space as the teacher's states. To ensure both teacher and student output pre-softmax attention logits, the forward pass of self-attention has been adapted accordingly.
- $L_{embd} = MSE(E^S W_e, E^T)$ is the embedding-based distillation. E^S and E^T refers to the student's and teacher's embeddings respectively, while W_e has the same role as W_h.

In this work, W_h and W_e were not treated as single matrices, but a separate linear projection $W^{(m)}$ for each matched representation has been learned.

To overcome the mismatch in the number of layers of student and teacher models, a mapping function $n = g(m)$ between student and teacher layers has been used. In this way, the $m-th$ layer of student model learns from the $g(m)-th$ layer of teacher model.

The hyperparameters used for this subphase were the following:

- Epochs: 8
- Batch size: 32 (physical dim = 8 and gradient accumulation = 4)
- Learning rate: $5e-5$

3.4.2 Phase 2: Prediction Layer Distillation

During this phase, only prediction layer distillation is performed in order to also fit the predictions of teacher model. A variant of prediction loss has been proposed here. Jiao et al. rely on the following loss defined by Hinton et al., which penalizes the soft cross-entropy loss between the student network's logits against the teacher's logits:

$$L_{pred} = CE(z^T/T, z^S/T) \tag{2}$$

However, by introducing the concept of imperfect teachers, it is possible to rewrite the loss by also introducing hard labels (ground truth) and weights for both soft and hard labels, so that the student learns from the teacher when the teacher produces correct predictions and learns to trust the teacher less when the teacher produces incorrect predictions.

The prediction layer loss implemented in this work has been the following:

$$L_{TD-2} = \rho(x) \cdot \underbrace{CE(z_T/T, z_S/T)}_{\text{Soft Loss}} \cdot T^2 + (1 - \rho(x)) \cdot \underbrace{WCE(z_S, y_{true})}_{\text{Weighted Hard Loss}} \tag{3}$$

where

- $\rho(x)$: adaptive coefficient. For this work, $\rho(x) = 0.9$ has been used if teacher was correct, $\rho(x) = 0.2$ otherwhise.
- z_T, z_S : teacher's and student's logits respectively.
- T : the temperature parameter. For this work, $T = 1$ worked fine.
- $WCE(z_S, y_{true})$: Weighted Cross Entropy Loss between student's logits and ground truth. As in teacher's fine tuning, the weight has been calculated on the train set as the ratio between No spoiler and Spoiler reviews.

The hyperparameters used for this subphase were the following:

- Epochs: 3
- Batch size: 32 (physical dim = 16 and gradient accumulation = 2)
- Learning rate: $2e - 5$
- Metric for best model: PR-AUC on validation set.

3.5 Results of distillation

As shown in this subsection, distillation achieved its goals.

3.5.1 Performance Drop

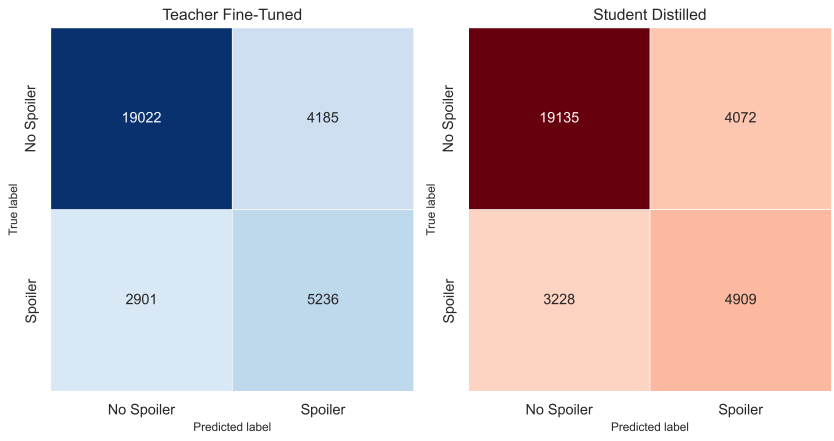

Figure 2: Inserisci qui la didascalia della tua figura.

Figure 2 and Figure show the confusion matrices of the fine-tuned teacher and the distilled student and bar charts comparing the evaluation metrics of the two models and the COTS (out-of-the-shelf) student respectively, all evaluated on the initially constructed test set. It can be seen that the performance of the two models is very similar, with the student introducing a slightly higher number of false negatives and true positives. The performance delta is very small for every metric, demonstrating the effective effectiveness of the KD. The COTS student has a recall of 1, but all other metrics are much lower. This indicates the model captures all actual positives, but at the cost of high false positives. This is expected with the imbalanced dataset and a model with few parameters, which learns to predict the positive class too liberally.

3.5.2 Computational Savings

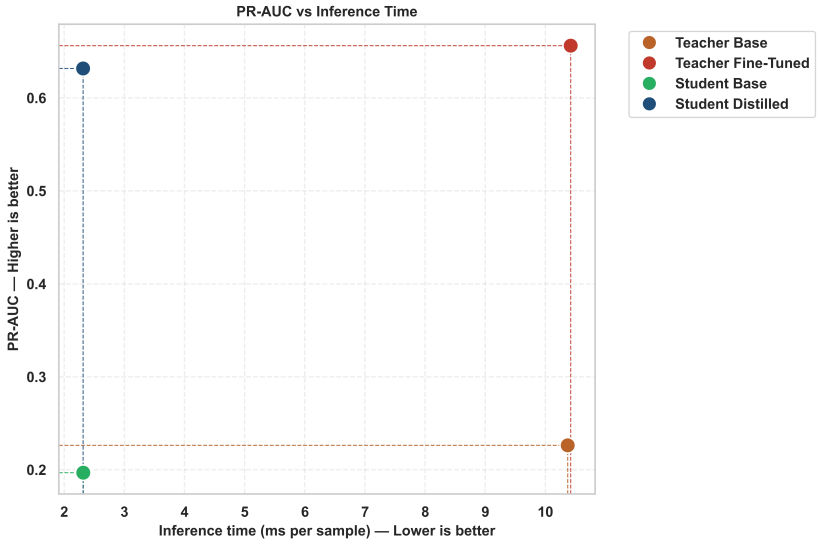

Figure 3: Inserisci qui la didascalia della tua figura.

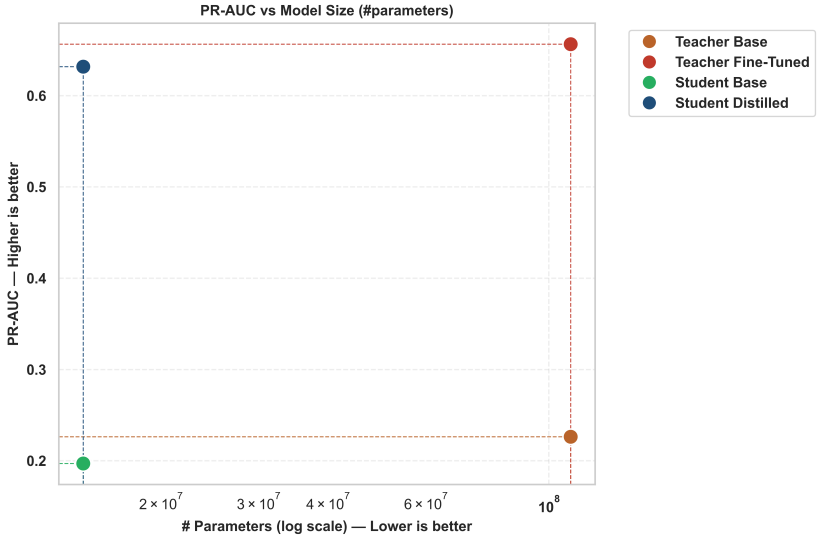

Figure 4: Inserisci qui la didascalia della tua figura.

Figure 3 and Figure 4 show two scatter plots: the first one shows the relationship between PR-AUC and inference time (ms per sample) and the second one shows the relationship between PR-AUC and model size (# of parameters) respectively. It can be seen, in both graphs, that the distilled student is in a more optimal point than the teacher (both base and fine-tuned), both in terms of inference time and model size and it outperformes the COTS student and not fine tuned teacher.

3.5.3 Interpretability

4 Conclusions

References

[1] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: pre-training of deep bidirectional transformers for language understanding. *CoRR*, abs/1810.04805, 2018.

[2] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network, 2015.

[3] Guangda Ji and Zhanxing Zhu. Knowledge distillation in wide neural networks: Risk bound, data efficiency and imperfect teacher, 2020.

[4] Xiaoqi Jiao, Yichun Yin, Lifeng Shang, Xin Jiang, Xiao Chen, Linlin Li, Fang Wang, and Qun Liu. Tinybert: Distilling bert for natural language understanding, 2020.

[5] Rishabh Misra. Imdb spoiler dataset, 05 2019.

[6] Chi Sun, Xipeng Qiu, Yige Xu, and Xuanjing Huang. How to fine-tune bert for text classification?, 2020.